Proyecto 6: Panel de Monitoreo en el SOC

Implementación de Zabbix Server 7.0 en Ubuntu 24.04

Descripción general de la infraestructura

He desplegado **Zabbix Server 7.0** sobre **Ubuntu 24.04 LTS** en una máquina virtual **VirtualBox**, con el objetivo de centralizar la supervisión de la infraestructura y disponer de un panel operativo para el seguimiento de disponibilidad, rendimiento y estado de servicios.

El alcance de la monitorización implementada comprende:

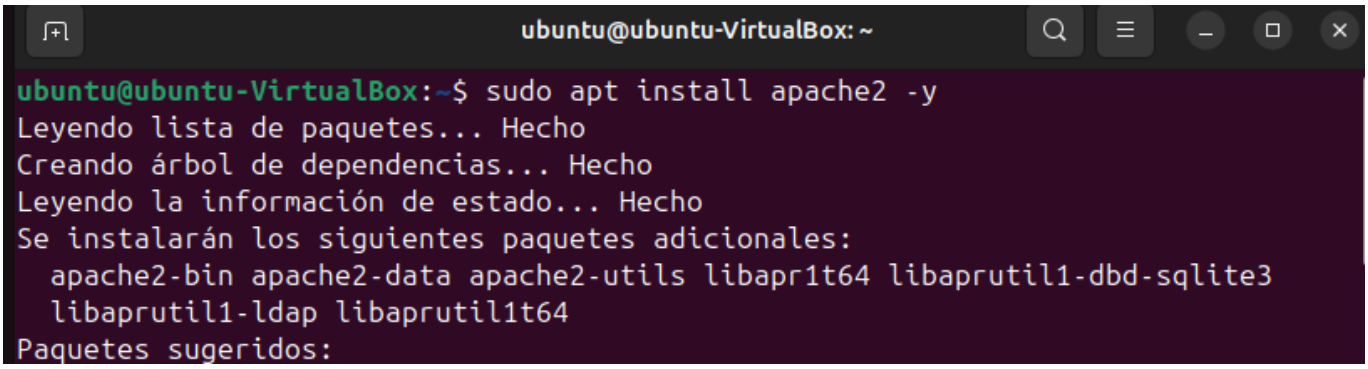
- Servidor Ubuntu local, nodo principal donde reside Zabbix Server.
- Máquina virtual Kali Linux, host cliente monitorizado mediante agente.

Este enfoque me ha permitido consolidar métricas, eventos y alertas en un único punto de control, facilitando la detección temprana de incidencias y el análisis de tendencias.

Fase 1: Preparación del entorno base

Actualización e instalación de componentes esenciales

He actualizado el sistema operativo e instalado las dependencias necesarias para el despliegue del stack completo de Zabbix (servidor, frontend web y base de datos). Esta fase me ha asegurado compatibilidad de paquetes, disponibilidad de librerías requeridas y estabilidad previa a la puesta en marcha.



```
ubuntu@ubuntu-VirtualBox: ~
ubuntu@ubuntu-VirtualBox:~$ sudo apt install php php-{cgi,common,mbstring,net-so
cket,gd,xml-util,mysql,bcmath,imap,snmp} -y
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Nota, seleccionando «php-pear» en lugar de «php-xml-util»
Se instalarán los siguientes paquetes adicionales:
  libapache2-mod-php8.3 libc-client2007e libsodium23 mlock php-xml php8.3
  php8.3-bcmath php8.3-cgi php8.3-cli php8.3-common php8.3-gd php8.3-imap
  php8.3-mbstring php8.3-mysql php8.3-opcache php8.3-readline php8.3-snmp
  php8.3-xml snmp
Paquetes sugeridos:
  uw-mailutils
Se instalarán los siguientes paquetes NUEVOS:
  libapache2-mod-php8.3 libc-client2007e libsodium23 mlock php php-bcmath
  php-cgi php-common php-gd php-imap php-mbstring php-mysql php-net-socket
  php-pear php-snmp php-xml php8.3 php8.3-bcmath php8.3-cgi php8.3-cli
  php8.3-common php8.3-gd php8.3-imap php8.3-mbstring php8.3-mysql
  php8.3-opcache php8.3-readline php8.3-snmp php8.3-xml snmp
0 actualizados, 30 nuevos se instalarán, 0 para eliminar y 0 no actualizados.
Se necesita descargar 8.954 kB de archivos.
Se utilizarán 41,3 MB de espacio de disco adicional después de esta operación.
Des:1 http://es.archive.ubuntu.com/ubuntu noble/main amd64 php-common all 2:93ub
untu2 [13,9 kB]
```

```
ubuntu@ubuntu-VirtualBox: ~
ubuntu@ubuntu-VirtualBox:~$ sudo apt install libapache2-mod-php -y
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes NUEVOS:
  libapache2-mod-php
0 actualizados, 1 nuevos se instalarán, 0 para eliminar y 0 no actualizados.
Se necesita descargar 4.224 B de archivos.
Se utilizarán 15,4 kB de espacio de disco adicional después de esta operación.
Des:1 http://es.archive.ubuntu.com/ubuntu noble/main amd64 libapache2-mod-php al
l 2:8.3+93ubuntu2 [4.224 B]
Descargados 4.224 B en 0s (25,2 kB/s)
Seleccionando el paquete libapache2-mod-php previamente no seleccionado.
(Leyendo la base de datos ... 152769 ficheros o directorios instalados actualmen
te.)
Preparando para desempaquetar .../libapache2-mod-php_2%3a8.3+93ubuntu2_all.deb .
..
Desempaquetando libapache2-mod-php (2:8.3+93ubuntu2) ...
Configurando libapache2-mod-php (2:8.3+93ubuntu2) ...
ubuntu@ubuntu-VirtualBox:~$
```

```
ubuntu@ubuntu-VirtualBox: ~
ubuntu@ubuntu-VirtualBox:~$ sudo apt install mysql-server -y
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes adicionales:
  libaio1t64 libcgi-fast-perl libcgi-pm-perl libevent-core-2.1-7t64
  libevent-pthreads-2.1-7t64 libfcgi-bin libfcgi-perl libfcgi0t64
  libhtml-template-perl libmecab2 libprotobuf-lite32t64 mecab-ipadic
  mecab-ipadic-utf8 mecab-utils mysql-client-8.0 mysql-client-core-8.0
  mysql-common mysql-server-8.0 mysql-server-core-8.0
Paquetes sugeridos:
  libipc-sharedcache-perl mailx tinyca
Se instalarán los siguientes paquetes NUEVOS:
  libaio1t64 libcgi-fast-perl libcgi-pm-perl libevent-core-2.1-7t64
  libevent-pthreads-2.1-7t64 libfcgi-bin libfcgi-perl libfcgi0t64
  libhtml-template-perl libmecab2 libprotobuf-lite32t64 mecab-ipadic
```

1.1 Configuración inicial de MySQL

He habilitado el servicio para arranque automático mediante el siguiente comando:

```
sudo systemctl enable mysql
```

```
ubuntu@ubuntu-VirtualBox: ~
ubuntu@ubuntu-VirtualBox:~$ sudo systemctl start mysql
ubuntu@ubuntu-VirtualBox:~$ sudo systemctl enable mysql
Synchronizing state of mysql.service with SysV service script with /usr/lib/syst
emd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable mysql
ubuntu@ubuntu-VirtualBox:~$ sudo systemctl status mysql
● mysql.service - MySQL Community Server
     Loaded: loaded (/usr/lib/systemd/system/mysql.service; enabled; preset: en>
     Active: active (running) since Thu 2026-02-26 17:26:46 CET; 1min 10s ago
   Main PID: 21157 (mysqld)
     Status: "Server is operational"
      Tasks: 37 (limit: 4600)
     Memory: 363.6M (peak: 377.8M)
        CPU: 909ms
     CGroup: /system.slice/mysql.service
             └─21157 /usr/sbin/mysqld

feb 26 17:26:46 ubuntu-VirtualBox systemd[1]: Starting mysql.service - MySQL Co>
feb 26 17:26:46 ubuntu-VirtualBox systemd[1]: Started mysql.service - MySQL Com>
lines 1-13/13 (END)
```

Con esta configuración, el servicio de base de datos queda habilitado para iniciarse automáticamente en cada reinicio, garantizando persistencia operativa y evitando paradas del backend de Zabbix por reinicios del

sistema.

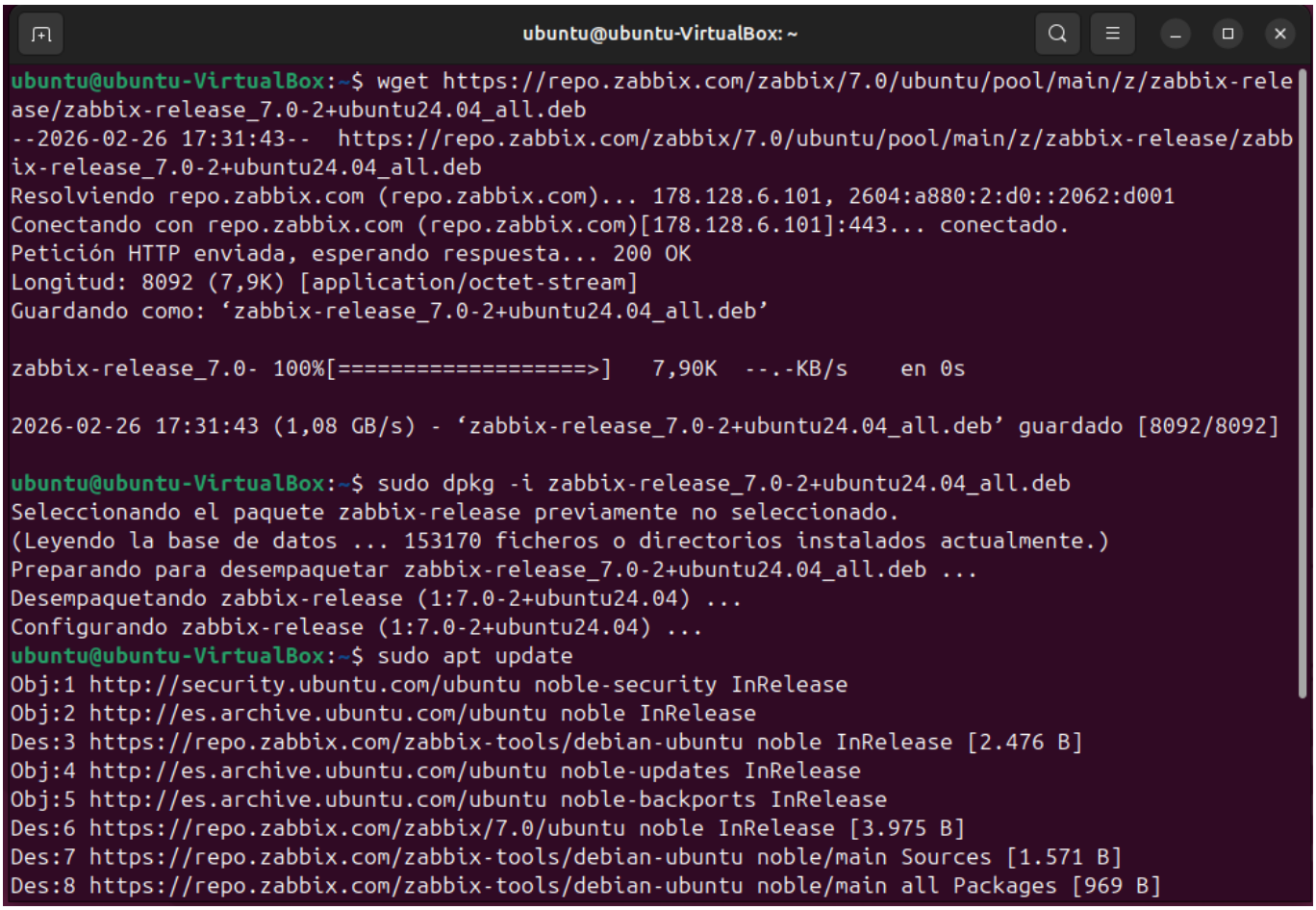
A continuación, he ajustado el siguiente parámetro para permitir la importación del esquema de Zabbix:

```
SET GLOBAL log_bin_trust_function_creators = 1;
```

Este parámetro facilita la creación de funciones y objetos necesarios durante la importación del esquema inicial en entornos donde MySQL aplica restricciones relacionadas con el binlog, evitando errores durante la carga inicial de la base de datos.

1.2 Instalación del repositorio oficial de Zabbix 7.0

He integrado el repositorio oficial de Zabbix para asegurar que los paquetes instalados (servidor, frontend y agentes) procedan de una fuente mantenida y con versiones alineadas entre sí. Esto reduce problemas de incompatibilidades y simplifica futuras actualizaciones.



```
ubuntu@ubuntu-VirtualBox: ~
ubuntu@ubuntu-VirtualBox:~$ sudo apt install zabbix-server-mysql zabbix-frontend-php zabbix-apache
-conf zabbix-sql-scripts zabbix-agent -y
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes adicionales:
  fonts-dejavu fonts-dejavu-extra fping libevent-extra-2.1-7t64 libmodbus5 libmysqlclient21
  libodbc2 libopenipmi0t64 php-curl php-ldap php8.3-curl php8.3-ldap snmpd
Paquetes sugeridos:
  odbc-postgresql tdsodbc snmptrapd zabbix-nginx-conf
Se instalarán los siguientes paquetes NUEVOS:
  fonts-dejavu fonts-dejavu-extra fping libevent-extra-2.1-7t64 libmodbus5 libmysqlclient21
  libodbc2 libopenipmi0t64 php-curl php-ldap php8.3-curl php8.3-ldap snmpd zabbix-agent
  zabbix-apache-conf zabbix-frontend-php zabbix-server-mysql zabbix-sql-scripts
0 actualizados, 18 nuevos se instalarán, 0 para eliminar y 0 no actualizados.
Se necesita descargar 22,1 MB de archivos.
Se utilizarán 77,8 MB de espacio de disco adicional después de esta operación.
Des:1 http://es.archive.ubuntu.com/ubuntu noble-updates/main amd64 snmpd amd64 5.9.4+dfsg-1.1ubunt
```

1.3 Creación y parametrización de la base de datos

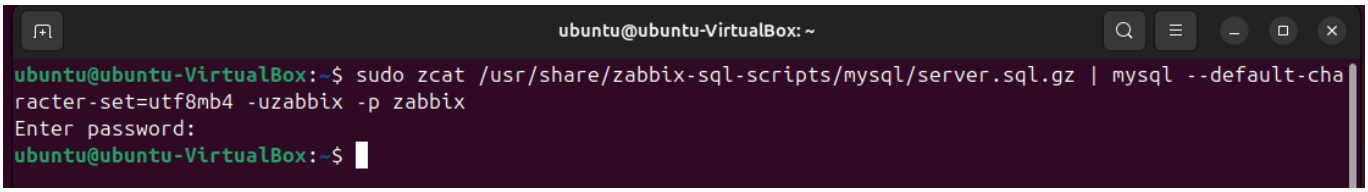
He creado una base de datos dedicada y un usuario específico para Zabbix, siguiendo un criterio de separación lógica y de seguridad. Los objetivos de esta configuración son:

- Mantener los datos de Zabbix aislados: tablas, históricos, eventos y configuración.
- Aplicar el principio de mínimo privilegio al usuario de base de datos.
- Garantizar trazabilidad y un mantenimiento más sencillo del sistema.

```
mysql> CREATE DATABASE zabbix CHARACTER SET utf8mb4 COLLATE utf8mb4_bin;
Query OK, 1 row affected (0,00 sec)

mysql> CREATE USER 'zabbix'@'localhost' IDENTIFIED BY '          ';
Query OK, 0 rows affected (0,02 sec)

mysql> GRANT ALL PRIVILEGES ON zabbix.* TO 'zabbix'@'localhost';
Query OK, 0 rows affected (0,01 sec)

mysql> SET GLOBAL log_bin_trust_function_creators = 1;
Query OK, 0 rows affected, 1 warning (0,00 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0,01 sec)
```

1.4 Importación del esquema inicial de Zabbix

He importado el esquema base de Zabbix en la base de datos recién creada. Esta operación genera la estructura necesaria (tablas, relaciones e información inicial) para que el servidor pueda almacenar métricas, registrar eventos y mantener el histórico operativo.



1.5 Configuración del servidor Zabbix

He editado el archivo principal de configuración:

```
sudo nano /etc/zabbix/zabbix_server.conf
```

Los parámetros críticos que he configurado son los siguientes:

```
DBHost=localhost
DBName=zabbix
DBUser=zabbix
DBPassword=[CREDENCIAL_SEGURA]
```

La finalidad de cada parámetro es la siguiente:

- **DBHost**: define el destino de la base de datos; en local mejora la latencia y simplifica el despliegue.
- **DBName**: identifica la base de datos que utilizará Zabbix para su persistencia.
- **DBUser** y **DBPassword**: credenciales del usuario dedicado, necesarias para que Zabbix lea y escriba métricas, eventos, configuración y registros de housekeeping.

Una configuración correcta en este archivo es imprescindible para que el servidor Zabbix arranque y opere de forma estable.

1.6 Activación e inicialización de servicios Zabbix

He reiniciado los servicios y configurado el inicio automático con los siguientes comandos:

```
sudo systemctl restart zabbix-server zabbix-agent2 apache2
sudo systemctl enable zabbix-server zabbix-agent2 apache2
```

Con esto he garantizado que:

- El servidor Zabbix esté activo para procesar checks, triggers y eventos.
- El agente local reporte métricas del propio servidor mediante auto-monitoreo.
- El servicio web (Apache) publique el frontend para administración y visualización.

He realizado la comprobación del estado de los servicios con los siguientes comandos:

```
sudo systemctl status zabbix-server
sudo systemctl status zabbix-agent2
```

La validación del estado ha confirmado que los servicios se encuentran en ejecución y que no existen errores de configuración ni dependencias sin resolver.

```
ubuntu@ubuntu-VirtualBox:~$ sudo systemctl restart zabbix-server
ubuntu@ubuntu-VirtualBox:~$ sudo systemctl restart zabbix-agent
ubuntu@ubuntu-VirtualBox:~$ sudo systemctl restart apache2
ubuntu@ubuntu-VirtualBox:~$ sudo systemctl status zabbix-server
● zabbix-server.service - Zabbix Server
     Loaded: loaded (/usr/lib/systemd/system/zabbix-server.service; disabled; preset: enabled)
     Active: active (running) since Thu 2026-02-26 17:54:00 CET; 37s ago
    Process: 24933 ExecStart=/usr/sbin/zabbix_server -c $CONFFILE (code=exited, status=0/SUCCESS)
   Main PID: 24935 (zabbix_server)
      Tasks: 1 (limit: 4600)
     Memory: 6.5M (peak: 8.1M)
        CPU: 15ms
     CGroup: /system.slice/zabbix-server.service
             └─24935 /usr/sbin/zabbix_server -c /etc/zabbix/zabbix_server.conf

feb 26 17:54:00 ubuntu-VirtualBox systemd[1]: Starting zabbix-server.service - Zabbix Server...
feb 26 17:54:00 ubuntu-VirtualBox systemd[1]: Started zabbix-server.service - Zabbix Server.
ubuntu@ubuntu-VirtualBox:~$
```

1.7 Configuración del agente local en Ubuntu

He editado el archivo del agente:

```
sudo nano /etc/zabbix/zabbix_agentd.conf
```

Los parámetros fundamentales configurados son:

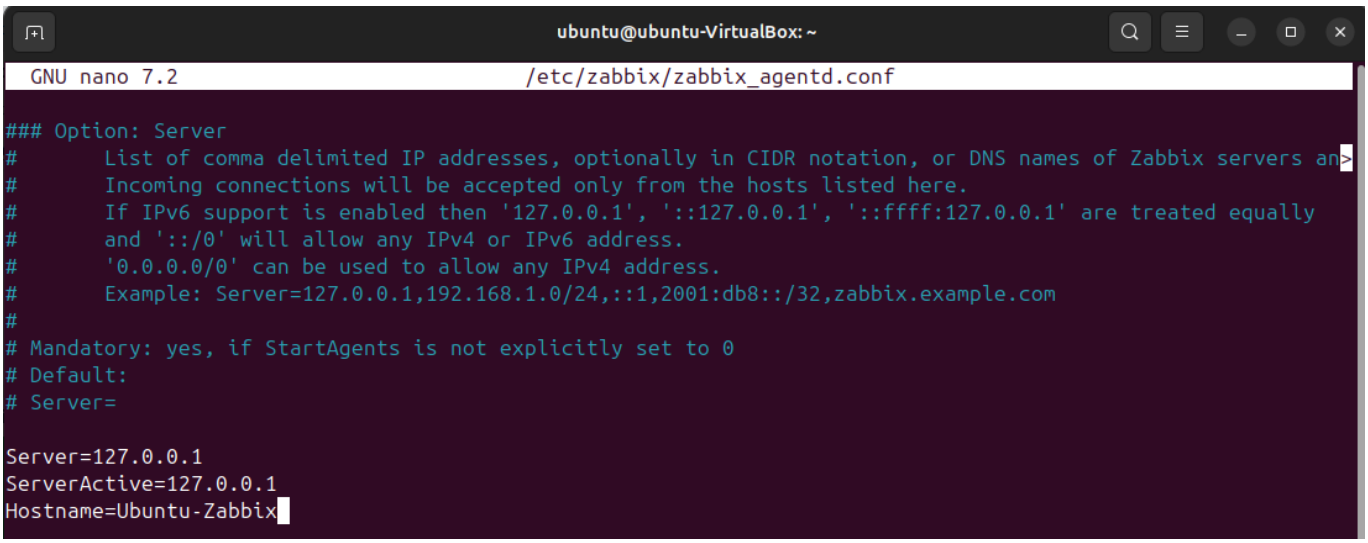
```
Server=127.0.0.1
ServerActive=127.0.0.1
Hostname=Ubuntu-Zabbix
```

La justificación de cada ajuste es la siguiente:

- **Server**: define qué servidor tiene permiso para consultar al agente en modo pasivo.
- **ServerActive**: especifica el servidor al que el agente enviará datos en modo activo, útil para escenarios con restricciones de red o para un mayor control del flujo de envío.
- **Hostname**: identificador lógico del host dentro de Zabbix; debe coincidir con el nombre registrado en el frontend para evitar incoherencias en el inventario.

Tras la edición, he reiniciado el agente:

```
sudo systemctl restart zabbix-agent2
```

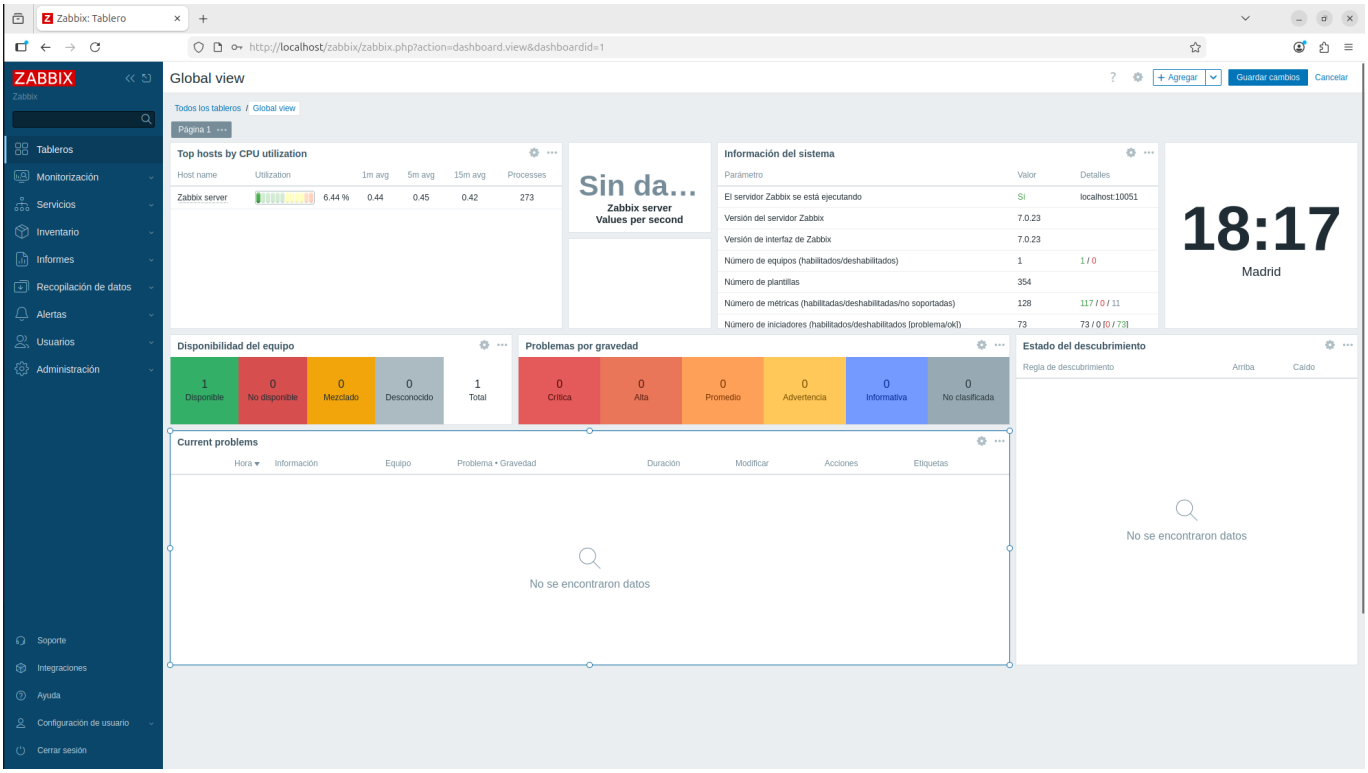


1.8 Finalización mediante interfaz web

He accedido al asistente de instalación a través del siguiente endpoint:

```
http://localhost/zabbix/setup.php
```

He completado el asistente web verificando los requisitos del entorno, la conectividad con la base de datos y los parámetros finales del frontend. Tras finalizar, he accedido al panel principal, quedando habilitada la gestión de hosts, plantillas, items, triggers y acciones automatizadas.



Fase 2: Configuración del cliente desde Kali Linux

He instalado el agente en el cliente con el siguiente comando:

```
sudo apt install zabbix-agent
```

He configurado el agente del cliente para comunicarse con el servidor Zabbix y reportar métricas del sistema: CPU, memoria, disco, procesos y conectividad, entre otras. Esta fase amplía el alcance del monitoreo hacia endpoints adicionales de la red. Para ello hemos configurado el fichero *zabbix-agentd.conf* aplicando la IP del Servidor zabbix y el hostname del cliente que posteriormente debemos de poner exactamente igual en el servidor.

```
kali@kali: ~
Session  Actions  Edit  View  Help
  GNU nano 8.7                    /etc/zabbix/zabbix_agentd.conf

Server=192.168.18.246
ServerActive=192.168.18.246
Hostname=Kali
```

He reiniciado el servicio y comprobado su estado:

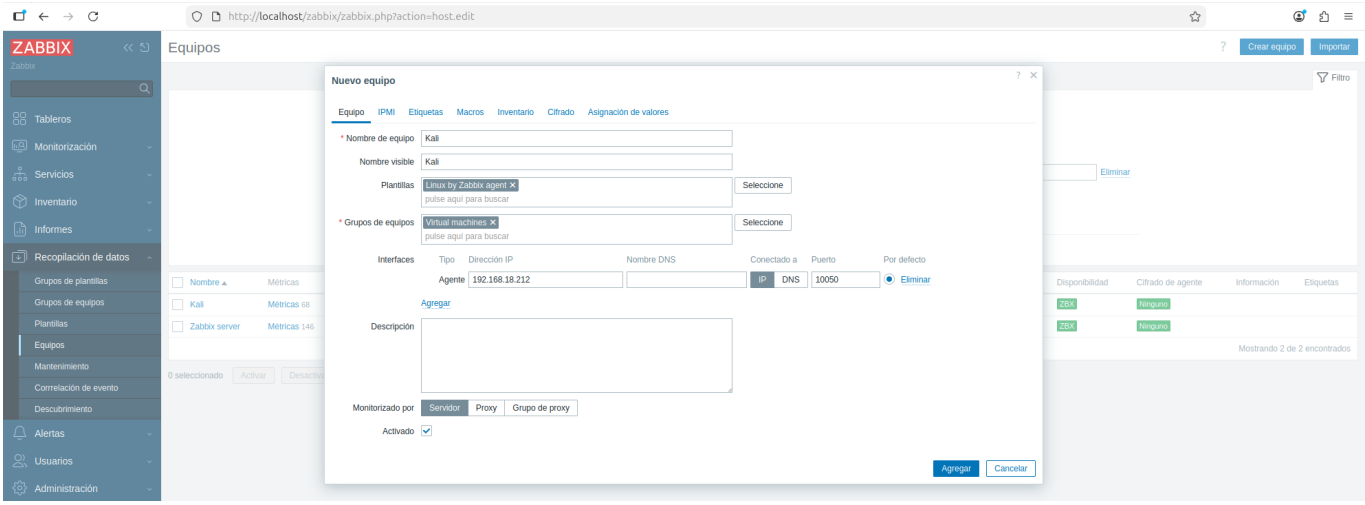
```
sudo systemctl restart zabbix-agent
sudo systemctl status zabbix-agent
```

```
┌──(kali㉿kali)-[~]
└─$ sudo systemctl status zabbix-agent
● zabbix-agent.service - Zabbix Agent
     Loaded: loaded (/usr/lib/systemd/system/zabbix-agent.service; enabled; preset: disabl>
     Active: active (running) since Thu 2026-02-26 12:58:32 EST; 17min ago
 Invocation: 08e23bf457df4d199085d5f5c716ba2a
       Docs: man:zabbix_agentd
   Main PID: 14805 (zabbix_agentd)
      Tasks: 13 (limit: 2119)
     Memory: 19.2M (peak: 24.7M)
        CPU: 885ms
     CGroup: /system.slice/zabbix-agent.service
             ├─14805 /usr/sbin/zabbix_agentd --foreground
             ├─14815 "/usr/sbin/zabbix_agentd: collector [idle 1 sec]"
             ├─14816 "/usr/sbin/zabbix_agentd: listener #1 [waiting for connection]"
             ├─14817 "/usr/sbin/zabbix_agentd: listener #2 [waiting for connection]"
             ├─14818 "/usr/sbin/zabbix_agentd: listener #3 [waiting for connection]"
             ├─14819 "/usr/sbin/zabbix_agentd: listener #4 [waiting for connection]"
             ├─14820 "/usr/sbin/zabbix_agentd: listener #5 [waiting for connection]"
             ├─14821 "/usr/sbin/zabbix_agentd: listener #6 [waiting for connection]"
             ├─14822 "/usr/sbin/zabbix_agentd: listener #7 [waiting for connection]"
             ├─14823 "/usr/sbin/zabbix_agentd: listener #8 [waiting for connection]"
             ├─14824 "/usr/sbin/zabbix_agentd: listener #9 [waiting for connection]"
             ├─14825 "/usr/sbin/zabbix_agentd: listener #10 [waiting for connection]"
             └─14826 "/usr/sbin/zabbix_agentd: active checks #1 [idle 1 sec]"

Feb 26 12:58:32 kali systemd[1]: Started zabbix-agent.service - Zabbix Agent.
Feb 26 12:58:33 kali zabbix_agentd[14805]: Starting Zabbix Agent [Kali]. Zabbix 7.0.22 (re>
Feb 26 12:58:33 kali zabbix_agentd[14805]: Press Ctrl+C to exit.
 ... skipping ...
● zabbix-agent.service - Zabbix Agent
```

Integración del cliente en el servidor

Para registrar el host cliente en el panel, he seguido el siguiente procedimiento:

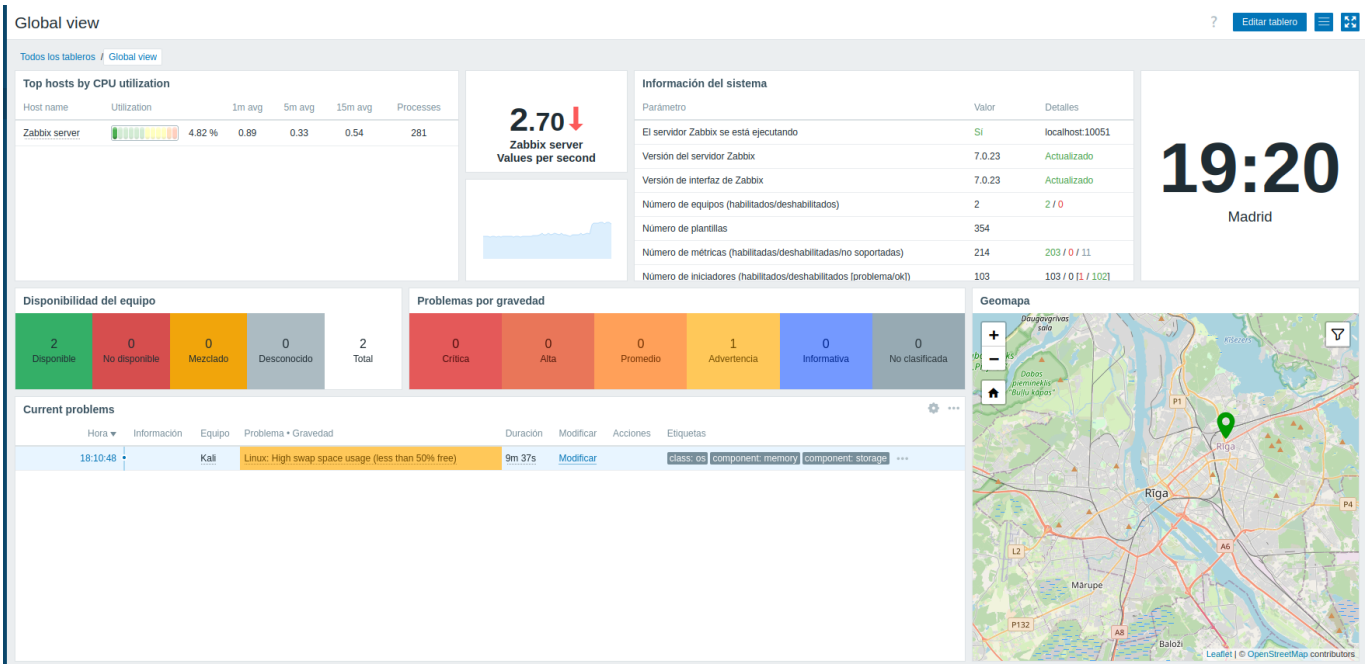
1. He accedido al menú **Recopilación de datos > Equipos**.
2. He seleccionado **Crear equipo**.
3. He completado los datos del cliente: nombre, interfaz del agente, grupo asignado y plantilla correspondiente.



Una vez registrado, el servidor ha comenzado a asociar métricas al host y se ha habilitado la visualización centralizada en el dashboard.

Verificación final

He confirmado la incorporación del cliente Kali Linux en el panel de Zabbix, verificando comunicación correcta y disponibilidad de datos en tiempo real. Con esto, la plataforma queda operativa y lista para ampliar el monitoreo a más hosts, definir umbrales de alerta personalizados y establecer acciones de notificación automatizadas.



Fase 3: Implementación de CheckMK para Monitoreo Multiplataforma

Instalación del servidor CheckMK en Ubuntu

He procedido a instalar **CheckMK Raw Edition** siguiendo la documentación oficial de la plataforma, ejecutando los comandos proporcionados en su repositorio oficial sobre **Ubuntu 24.04 LTS**. Este proceso automatizado ha desplegado el stack completo:

- Servidor web Apache con PHP optimizado
- Base de datos MariaDB/MySQL dedicada
- Core de monitoreo CheckMK con agente integrado
- Interfaz web multilingüe accesible por HTTPS

La instalación se ha completado sin incidencias, quedando disponible el panel de administración en `https://checkmk.com/download`.

checkmk Product Solutions Pricing Learn Community Contact Sales

3 Install Checkmk

3.1 Download Checkmk for Ubuntu or Debian

Pull in the package using *wget*

```
$ wget https://download.checkmk.com/checkmk/2.4.0p22/check-mk-cloud-2.4.0p22_0.noble_amd64.deb
```

Copy

Planning to install Checkmk on another machine? Download via browser View changelog

By downloading Checkmk you agree to the End User License Agreement and General Terms & Conditions applicable to your Checkmk Edition, accessible at https://checkmk.com/legal. You also agree to the Checkmk export control regulations, in particular you will not pass on the software to persons or companies based in Iran, on Crimea, in Cuba, North Korea, Russia, Sudan or Syria.

SHA-256 file hash:

```
a80c9753852f11f5d59882277cb25867068738a1a006ae2b8cd5489ec655c562
```

Copy

3.2 Signed-package installation

All packages are signed using GnuPG. Through the use of this signature, on the one hand it can be verified whether the package really is from Checkmk, and on the other hand it can be verified that the package is complete. Follow the steps described here.

3.3 Install the Checkmk package

Now install the package including all of its dependencies.

```
$ sudo apt install ./check-mk-cloud-2.4.0p22_0.noble_amd64.deb
```

Copy

Afterwards we can test if the installation was successful by running the *omd version* command:

```
$ omd version
```

Copy

```
OMD - Open Monitoring Distribution Version 2.4.0p22.cce
```

You should see an output similar to this one.

checkmk Product Solutions Pricing Learn Community Contact Sales

3.4 Create a Checkmk monitoring site

Use the *omd* command to create a new Checkmk site. You can choose your own name, in this example we named the site 'monitoring'

```
$ sudo omd create monitoring
```

Copy

After the site has been created you will see an output similar to the one below.

```
Output
Adding /opt/omd/sites/monitoring/tmp to /etc/fstab.
Creating temporary filesystem /omd/sites/monitoring/tmp...OK
Restarting Apache...OK
Created new site monitoring with version 2.4.0p22.cce.

    The site can be started with omd start monitoring.
    The default web UI is available at http://your_server/monitoring/

    The admin user for the web applications is cmkadmin with password: generated-password
    For command line administration of the site, log in with 'omd su monitoring'.
    After logging in, you can change the password for cmkadmin with 'cmk-passwd cmkadmin'.
```

Here you can see the URL on which your site is accessible, default username, and an automatically generated password for accessing the site. The site is now created, but it still needs to be started. To start the site, type:

```
$ sudo omd start monitoring
```

Copy

When everything has been started successfully you will see an output similar to the one below.

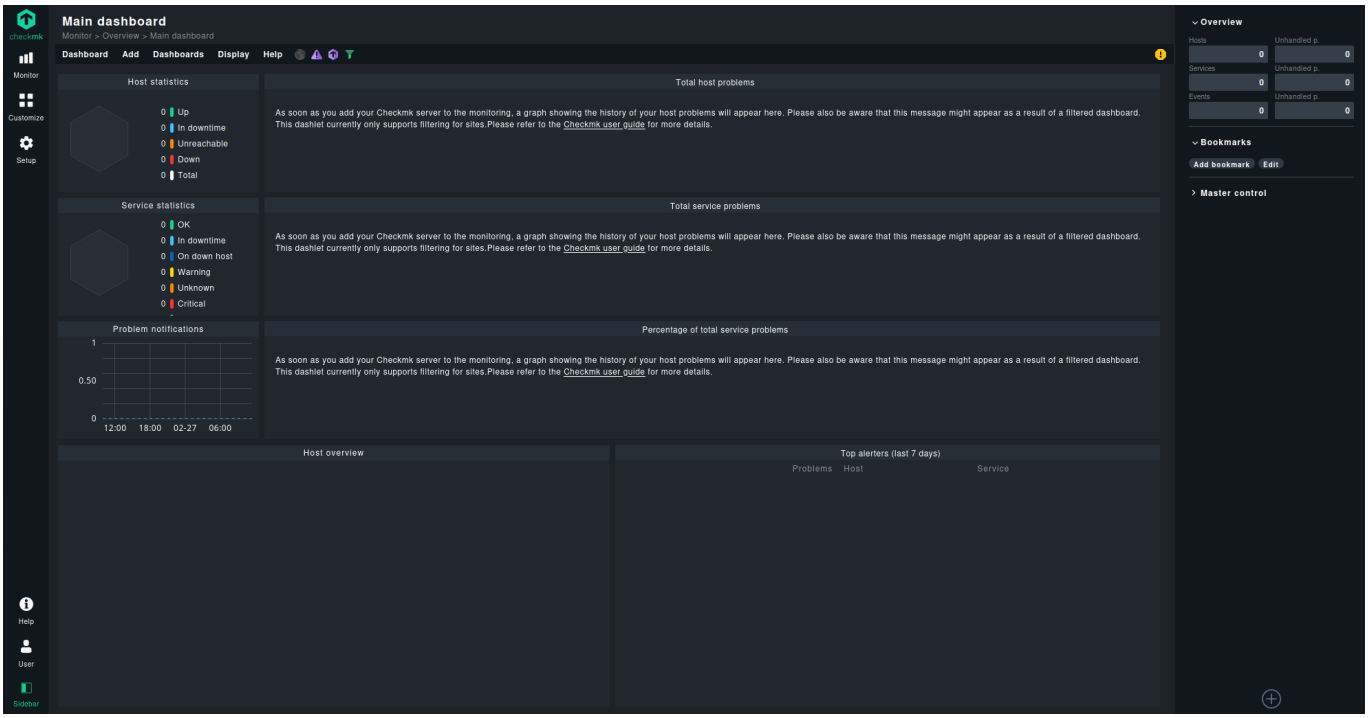
```
Starting mkeventd...OK
Starting liveproxyd...OK
Starting mknotifyd...OK
Starting rrdcached...OK
Starting cmc...OK
Starting apache...OK
Starting dcd...OK
Starting redis...OK
Initializing Crontab...OK
```

Your Checkmk site is now up and running. You can visit the user interface in the browser by using the aforementioned URL.

- Podemos ver que se a instalado todo correctamente siugiendo los pasos de la web oficial

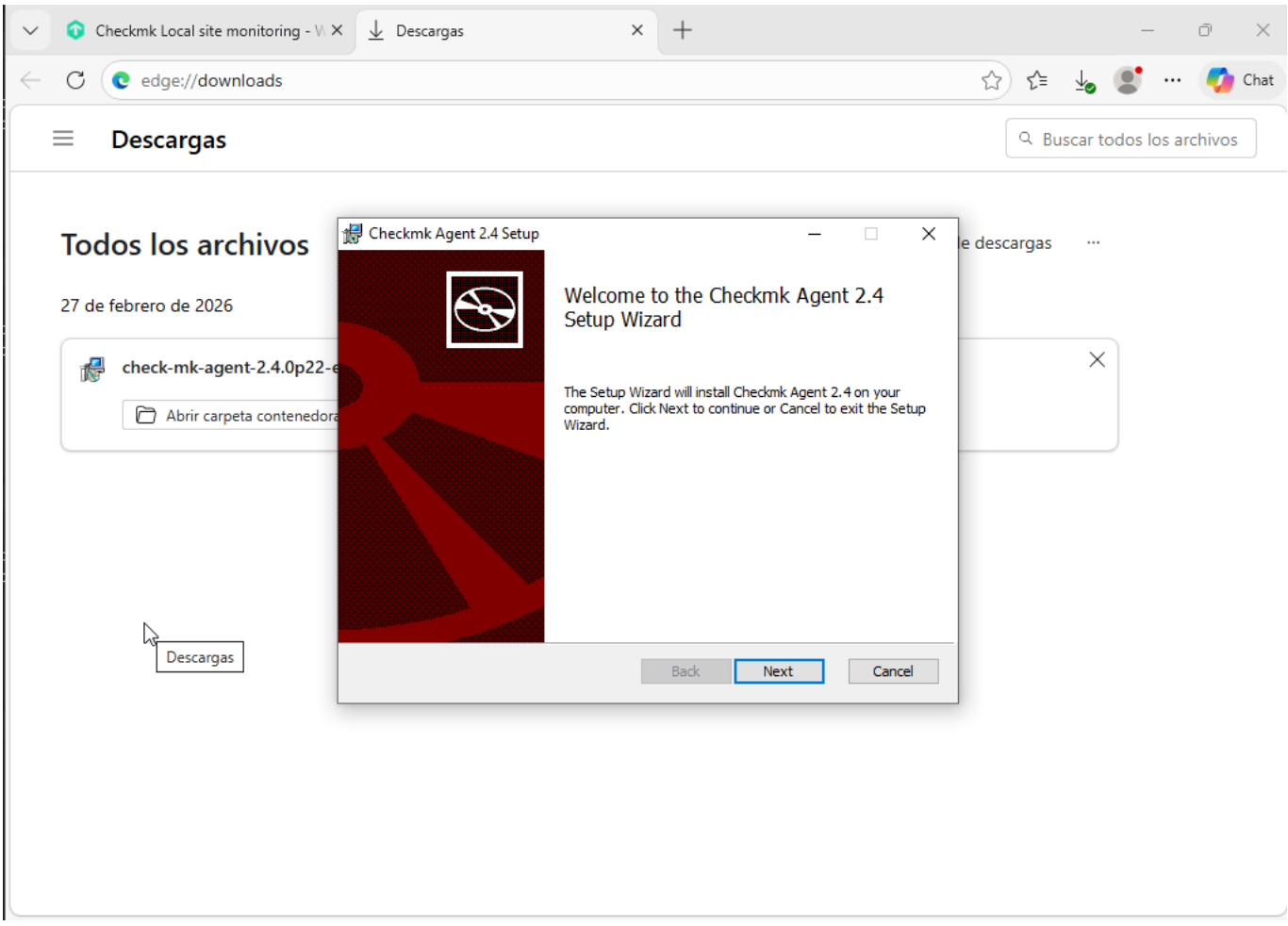
```
ubuntu@ubuntu-VirtualBox:~$ sudo omd create monitoring
Adding /opt/omd/sites/monitoring/tmp to /etc/fstab.
Creating temporary filesystem /omd/sites/monitoring/tmp...OK
Updating core configuration...
Generating configuration for core (type cmc)...
Starting full compilation for all hosts
 Creating global helper config...OK
 Creating cmc protobuf configuration...OK
Executing post-create script "01_create-sample-config.py"...OK
Executing post-create script "02_cmk-compute-api-spec"...OK
Executing post-create script "03_message-broker-certs"...OK
Restarting Apache...OK
Created new site monitoring with version 2.4.0p22.cce.
```

- Comprobamos finalmente en --> http://virtual-ubuntu/monitoring/



1.9 Configuración del agente CheckMK en cliente Windows 10

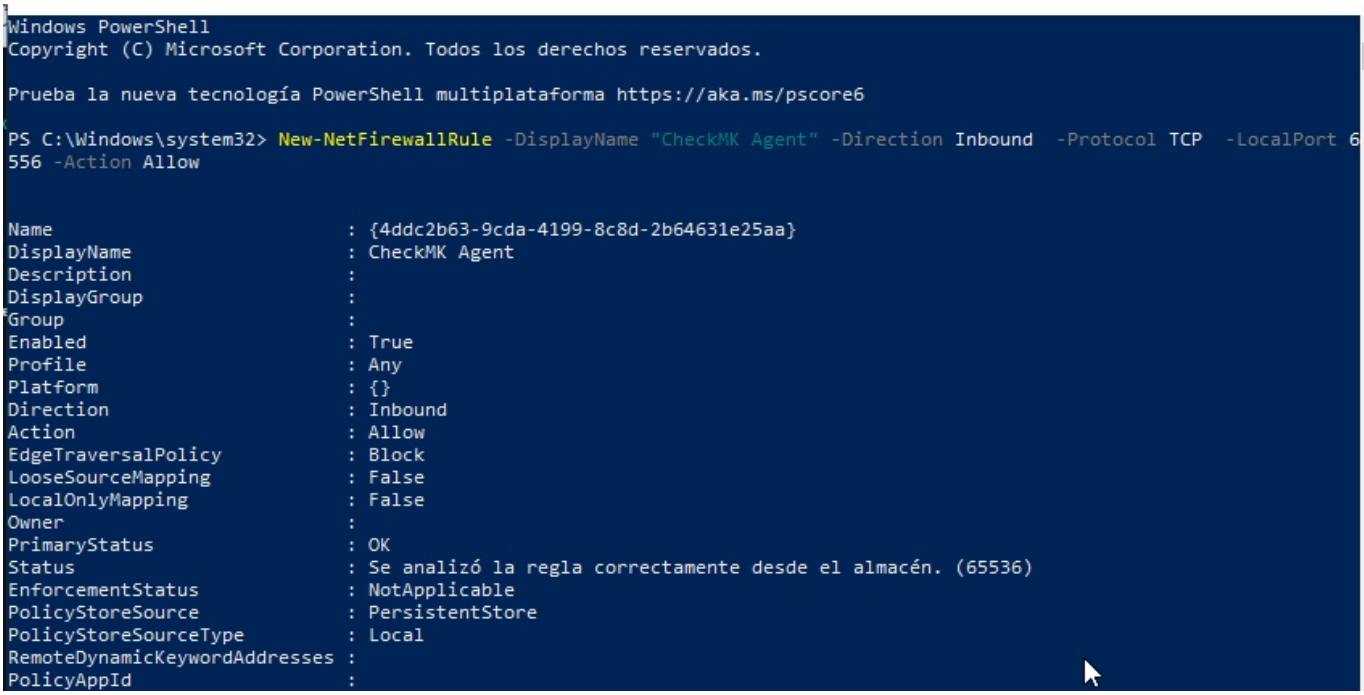
He descargado el instalador oficial **MSI** del agente CheckMK directamente desde el panel web del servidor recién instalado. En el equipo cliente **Windows 10**, he ejecutado el instalador con privilegios de administrador, configurando el servicio para escuchar en el puerto TCP **6556**.



Desafío identificado: Windows Defender Firewall rechazaba las conexiones entrantes al puerto del agente aunque estuviera explícitamente permitido. Para resolver esta restricción, he procedido a:

1. **Deshabilitar temporalmente Windows Defender** durante la fase de pruebas
2. **Crear regla de firewall específica** para puerto 6556 TCP (Inbound/Any Profile)
3. **Reiniciar el servicio Check MK Agent**

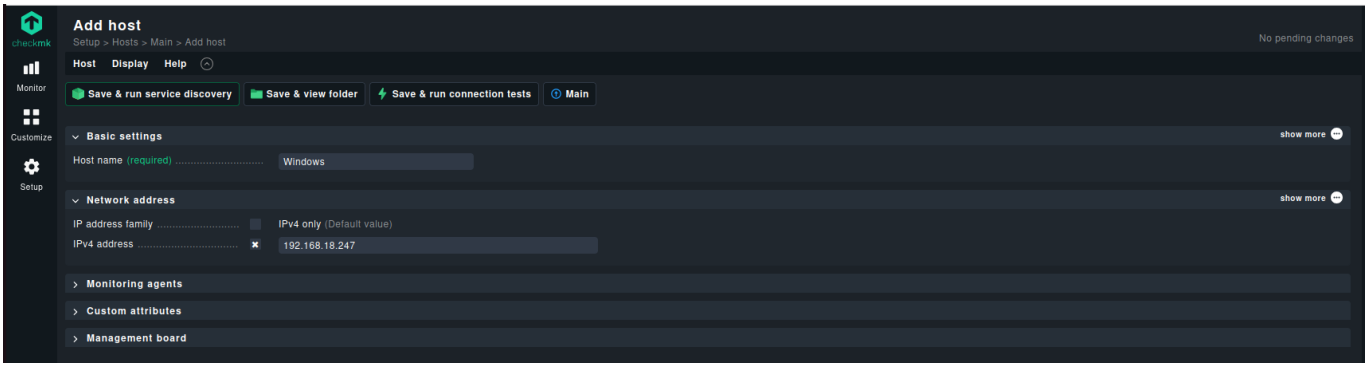
```
# Regla de firewall creada manualmente
New-NetFirewallRule -DisplayName "CheckMK Agent" -Direction Inbound -Protocol TCP
-LocalPort 6556 -Action Allow
```



Integración del host Windows 10 en CheckMK

En el panel web de CheckMK he seguido el procedimiento estándar de incorporación:

- Setup → Hosts → Add host
- Hostname: Nombre real del equipo Windows 10 (obtenido con comando hostname)
- IP Address: 192.168.18.247
- Agent type: CheckMK Agent (modo Pull por defecto)



Verificación operativa del monitoreo Windows

Confirmamos la conectividad completa desde el servidor CheckMK:

Comparativa de Sistemas de Monitorización

Zabbix 7.0 vs CheckMK Raw Edition

Análisis comparativo de ambas plataformas implementadas en el SOC, evaluando 10 características técnicas clave.

Característica	Zabbix 7.0	CheckMK Raw Edition
Modelo de despliegue	Servidor dedicado con agentes activos/pasivos. Stack manual: MySQL + Apache + PHP + Zabbix Server.	Appliance todo-en-uno. Instalación automatizada con stack integrado (Apache, MariaDB, core propio).
Sistema operativo objetivo	Linux (Ubuntu, CentOS, Debian). Agentes disponibles también para Windows, macOS y otros SO.	Linux (servidor). Agentes nativos para Windows, Linux y dispositivos de red con SNMP/IPMI.
Plataformas cliente monitorizadas	Kali Linux monitorizado vía `zabbix-agent2`. Configuración manual del archivo `agentd.conf`.	Windows 10 vía agente MSI descargado desde el panel web. Configuración guiada desde la interfaz.
Protocolo de comunicación	Puerto TCP 10050 (pasivo) y 10051 (activo). Configuración de `Server` y `ServerActive` en el agente.	Puerto TCP 6556. Modo Pull por defecto. Requiere apertura manual de regla en Windows Firewall.
Interfaz de administración	Frontend web PHP accesible en `http://localhost/zabbix`. Asistente de instalación `setup.php` integrado.	Panel web HTTPS en `http://[host]/monitoring/`. Interfaz multilingüe más moderna con flujo de trabajo guiado.
Configuración de hosts	Manual: crear equipo en *Recopilación de datos*, definir interfaz, grupo y plantilla desde el frontend.	Guiada: *Setup → Hosts → Add host*. Detección automática de servicios tras activar el host.

Característica	Zabbix 7.0	CheckMK Raw Edition
Base de datos backend	MySQL/MariaDB configurada manualmente. Requiere importación del esquema SQL inicial y ajuste de `log_bin_trust_function_creators`.	MariaDB integrada y gestionada automáticamente por el instalador. Sin intervención manual en el esquema.
Complejidad de instalación	**Alta**: múltiples pasos manuales (repositorio, BD, usuario, esquema, configuración de archivos `.conf`).	**Baja-Media**: un único script/instalador oficial que despliega todo el stack automáticamente.
Seguridad y acceso	Usuario de BD con mínimo privilegio, credenciales en `zabbix_server.conf`. Acceso HTTP por defecto.	Acceso por HTTPS nativo. Panel con roles y autenticación gestionados desde la propia interfaz web.
Escalabilidad y extensibilidad	Alta: soporta miles de hosts, proxies, reglas de descubrimiento automático y triggers avanzados.	Alta: soporte de múltiples sitios, *distributed monitoring* y API REST para integración con terceros.
Datos de rendimiento del host	CPU (uso, carga, cores), memoria RAM/swap, disco (espacio, I/O, latencia), red (tráfico entrante/saliente, errores, paquetes perdidos), procesos activos y uptime del sistema.	CPU (uso por core, load average), memoria RAM/swap/virtual, disco (uso, I/O, inodes), interfaces de red (tráfico, errores, estado), servicios en ejecución, temperatura y hardware (vía SNMP/IPMI).
Datos de disponibilidad del host	Estado del agente (disponible/no disponible/desconocido), latencia ICMP (ping), pérdida de paquetes, historial de disponibilidad y SLA calculado por periodos de tiempo.	Estado del host (UP/DOWN/UNREACH), latencia de respuesta del agente, historial de estados con gráficas de disponibilidad y cálculo de tiempo de inactividad acumulado.
Datos de servicios y aplicaciones	Monitorización de puertos TCP/UDP, servicios web (HTTP/HTTPS con códigos de respuesta), bases de datos, SSH, FTP, SMTP y checks personalizados vía scripts o Zabbix sender.	Checks de servicios nativos (HTTP, TCP, LDAP, DNS, SMTP), monitorización de logs, checks de certificados SSL, inventario automático de software instalado y detección de cambios en configuración.
Visualización y alertas	Dashboards personalizables con widgets (gráficas, mapas de red, top N hosts), triggers con niveles de severidad (info, warning, average, high, disaster) y acciones de notificación por email, script o webhook.	Vistas de servicio agrupadas por host/folder, gráficas de tendencia integradas (PNP4Nagios/RRDtool), sistema de notificaciones con reglas de escalado y panel de eventos con historial detallado.

Conclusión

Zabbix 7.0 ofrece mayor control granular y flexibilidad para entornos complejos, a costa de una configuración más laboriosa. **CheckMK** destaca por su instalación simplificada, interfaz más intuitiva y

despliegue acelerado de agentes en plataformas heterogéneas como Windows. Ambas soluciones son robustas para un SOC, siendo la elección dependiente del equilibrio entre control técnico y velocidad de despliegue.